

Uni-app共用分类列表页面：我的下载与评分功能实现

一、需求核心与整体设计

1.1 需求明确

基于现有共用分类列表页面，新增两大核心功能：

- **我的下载**：支持用户下载列表中的资源（如文档、视频、图片等），记录下载状态（未下载/下载中/已下载），并在“我的下载”模块中展示已下载资源，支持离线查看与删除。
- **评分功能**：允许用户对列表项进行星级评分（1-5星），实时展示当前资源的平均评分及用户个人评分记录，评分后同步至服务端并更新页面展示。

1.2 设计原则

- **共用性优先**：功能嵌入不破坏原有分类列表的筛选、排序、分页逻辑，适配不同分类（如课程、资料、工具）的资源类型。
- **状态可视化**：通过图标、文字、进度条等清晰反馈下载状态；评分使用星级组件直观展示，区分“个人评分”与“平均评分”。
- **离线兼容**：下载资源本地缓存，支持无网络时查看；评分记录本地临时存储，网络恢复后自动同步。
- **性能优化**：批量处理下载任务，避免重复下载；评分数据本地缓存，减少接口请求频次。

1.3 技术栈适配

沿用原有技术体系，补充核心依赖：

- 框架：Uni-app（多端适配，重点兼容微信小程序、App端、H5）
- 网络请求：封装的uni.request（统一拦截、token携带）
- 本地存储：uni.setStorageSync/uni.getStorageSync（存储下载记录、评分缓存）、uni.getFileSystemManager（管理本地下载文件）
- UI组件：Uni-app内置组件+自定义星级评分组件
- 下载管理：uni.downloadFile（资源下载）、uni.openDocument（打开下载文件）

二、核心功能实现方案

2.1 基础准备：共用列表页面结构复用

原有分类列表页面核心结构为“筛选栏+列表容器+分页加载”，新增功能需嵌入列表项内部，不改动外层框架。列表项数据格式扩展如下（新增下载与评分相关字段）：

Code block

```
1  // 列表项数据结构（扩展后）
2  {
3    id: 'res001', // 资源唯一标识
4    title: 'Uni-app开发实战手册', // 资源标题
5    category: '开发资料', // 分类
6    cover: '/static/covers/book1.png', // 封面图
7    size: '2.3MB', // 资源大小（用于下载展示）
8    fileUrl: 'https://api.example.com/files/uni-app-book.pdf', // 下载地址
9    // 评分相关字段
10   avgRating: 4.5, // 平均评分
11   ratingCount: 126, // 评分人数
12   myRating: 5, // 当前用户评分（未评则为0）
13   // 下载相关字段
14   downloadStatus: 2, // 0=未下载 1=下载中 2=已下载
15   downloadProgress: 100, // 下载进度（0-100）
16   localPath: '/storage/emulated/0/Android/data/xxx/files/uni-app-book.pdf' //
    本地缓存路径
17 }
```

2.2 我的下载功能实现

2.2.1 下载状态管理

通过页面data维护列表项的下载状态，结合本地存储记录已下载资源，确保页面刷新后状态不丢失。核心逻辑如下：

Code block

```
1  // pages/category-list/category-list.vue
2  export default {
3    data() {
4      return {
5        categoryData: [], // 分类列表数据
6        downloadTasks: {} // 下载任务对象，key为资源id，value为下载任务实例
7      };
8    },
9    onLoad(options) {
10     this.getCategoryList(options.categoryId); // 加载分类列表
11     this.syncDownloadStatus(); // 同步本地下载状态
```

```

12     },
13     methods: {
14         // 同步本地存储的下载状态到列表数据
15         syncDownloadStatus() {
16             const downloadedList = uni.getStorageSync('downloadedResources') || [];
17             this.categoryData = this.categoryData.map(item => {
18                 const downloadedItem = downloadedList.find(res => res.id === item.id);
19                 if (downloadedItem) {
20                     return { ...item, ...downloadedItem };
21                 }
22                 return item;
23             });
24         },
25         // 下载资源
26         downloadResource(item) {
27             // 避免重复下载
28             if (item.downloadStatus === 1 || item.downloadStatus === 2) {
29                 item.downloadStatus === 2 && uni.showToast({ title: '已下载, 可在我的下载
中查看', icon: 'none' });
30                 return;
31             }
32
33             // 创建下载任务
34             const downloadTask = uni.downloadFile({
35                 url: item.fileUrl,
36                 filePath:
` ${uni.env.USER_DATA_PATH}/${item.title}${item.fileUrl.slice(item.fileUrl.lastIndexOf('.'))}`, // 自定义存储路径
37                 success: (res) => {
38                     if (res.statusCode === 200) {
39                         // 下载成功: 更新状态+存入本地
40                         item.downloadStatus = 2;
41                         item.localPath = res.filePath;
42                         this.updateDownloadStorage(item);
43                         uni.showToast({ title: '下载完成' });
44                     }
45                 },
46                 fail: (err) => {
47                     item.downloadStatus = 0;
48                     item.downloadProgress = 0;
49                     uni.showToast({ title: '下载失败, 请重试', icon: 'none' });
50                     console.error('下载失败: ', err);
51                 }
52             });
53
54             // 监听下载进度
55             downloadTask.onProgressUpdate((progress) => {

```

```

56         item.downloadStatus = 1;
57         item.downloadProgress = progress.progress;
58         // 实时更新列表渲染
59         this.$forceUpdate();
60     });
61
62     // 存储任务实例，用于暂停/取消
63     this.downloadTasks[item.id] = downloadTask;
64 },
65 // 更新本地下载存储
66 updateDownloadStorage(item) {
67     const downloadedList = uni.getStorageSync('downloadedResources') || [];
68     const existIndex = downloadedList.findIndex(res => res.id === item.id);
69     if (existIndex > -1) {
70         downloadedList[existIndex] = { id: item.id, downloadStatus: 2,
localPath: item.localPath };
71     } else {
72         downloadedList.push({ id: item.id, downloadStatus: 2, localPath:
item.localPath });
73     }
74     uni.setStorageSync('downloadedResources', downloadedList);
75 },
76 // 取消下载
77 cancelDownload(item) {
78     const task = this.downloadTasks[item.id];
79     task && task.abort();
80     item.downloadStatus = 0;
81     item.downloadProgress = 0;
82     delete this.downloadTasks[item.id];
83 },

```

2.2.2 下载状态UI渲染

在列表项中嵌入下载状态展示区域，根据不同状态显示不同内容（按钮/进度条/已下载图标）：

Code block

```

1  <!-- 列表项模板 -->
2  <view class="resource-item" v-for="(item, index) in categoryData"
:key="item.id">
3      <image class="item-cover" :src="item.cover" mode="aspectFill"></image>
4      <view class="item-info">
5          <view class="item-title">{{ item.title }}</view>
6          <view class="item-meta">{{ item.category }} · {{ item.size }}</view>
7
8          <rating-component :avg-rating="item.avgRating" :my-rating="item.myRating"
@rate="handleRate(item)"></rating-component>
9      </view>

```

```

10
11     <view class="download-area">
12
13         <button v-if="item.downloadStatus === 0" @click="downloadResource(item)"
size="mini">下载</button>
14
15         <view v-else-if="item.downloadStatus === 1" class="downloading">
16             <progress :percent="item.downloadProgress" stroke-width="4"
activeColor="#007aff" />
17             <button @click="cancelDownload(item)" size="mini" type="warn">取消
</button>
18         </view>
19
20         <view v-else class="downloaded">
21             <icon type="success" size="20" color="#00cc66" />
22             <button @click="openDownloadedFile(item)" size="mini">打开</button>
23         </view>
24     </view>
25 </view>

```

2.2.3 我的下载模块联动

新增“我的下载”页面，读取本地存储的下载记录，展示已下载资源并支持操作：

Code block

```

1  <!-- pages/my-download/my-download.vue -->
2  <template>
3      <view class="my-download">
4          <view class="empty" v-if="downloadedList.length === 0">
5              <icon type="download" size="60" color="#ccc" />
6              <view class="empty-text">暂无已下载资源</view>
7          </view>
8          <view class="downloaded-item" v-for="item in downloadedList"
:key="item.id">
9              <view class="item-info" @click="openDownloadedFile(item)">
10                 <image :src="item.cover" mode="aspectFill"></image>
11                 <view class="item-detail">
12                     <view class="item-title">{{ item.title }}</view>
13                     <view class="item-size">{{ item.size }}</view>
14                 </view>
15             </view>
16             <button @click="deleteDownloadedFile(item)" size="mini" type="warn">删除
</button>
17         </view>
18     </view>
19 </template>

```

```
20
21 <script>
22 export default {
23   data() {
24     return {
25       downloadedList: [],
26       categoryOriginData: [] // 分类列表原始数据，用于同步删除状态
27     };
28   },
29   onLoad() {
30     this.getDownloadedList();
31     this.getCategoryOriginData();
32   },
33   methods: {
34     // 获取本地下载列表
35     getDownloadedList() {
36       const downloadedList = uni.getStorageSync('downloadedResources') || [];
37       // 关联资源基本信息（从分类接口获取完整数据）
38       this.downloadedList = downloadedList.map(downloadItem => {
39         const originItem = this.categoryOriginData.find(res => res.id ===
downloadItem.id);
40         return { ...originItem, ...downloadItem };
41       });
42     },
43     // 打开已下载文件
44     openDownloadedFile(item) {
45       uni.openDocument({
46         filePath: item.localPath,
47         showMenu: true,
48         success: (res) => console.log('打开文件成功'),
49         fail: (err) => uni.showToast({ title: '打开失败，请重试', icon: 'none' })
50       });
51     },
52     // 删除已下载文件（本地文件+存储记录）
53     deleteDownloadedFile(item) {
54       uni.showModal({
55         title: '提示',
56         content: '确定删除该下载资源吗？',
57         success: (res) => {
58           if (res.confirm) {
59             // 1. 删除本地文件
60             const fs = uni.getFileSystemManager();
61             fs.unlink({
62               filePath: item.localPath,
63               success: () => {
64                 // 2. 删除存储记录
```

```

65         let downloadedList = uni.getStorageSync('downloadedResources')
        || [];
66         downloadedList = downloadedList.filter(res => res.id !==
        item.id);
67         uni.setStorageSync('downloadedResources', downloadedList);
68         // 3. 同步分类列表状态
69         this.syncCategoryDownloadStatus(item.id);
70         // 4. 更新页面
71         this.getDownloadedList();
72         uni.showToast({ title: '删除成功' });
73     },
74     fail: (err) => {
75         uni.showToast({ title: '删除失败', icon: 'none' });
76         console.error('删除文件失败: ', err);
77     }
78 });
79 }
80 }
81 });
82 },

```

2.3 评分功能实现

2.3.1 自定义星级评分组件

封装可复用的星级评分组件，支持展示平均评分和用户个人评分，点击星级触发评分提交：

Code block

```

1  <!-- components/rating-component/rating-component.vue -->
2  <template>
3      <view class="rating-container">
4          <view class="rating-stars">
5              <icon
6                  v-for="(star, index) in 5"
7                  :key="index"
8                  :type="index < Math.ceil(avgRating) ? 'star' : 'star-o'"
9                  :color="index < Math.ceil(avgRating) ? '#ffcc33' : '#ccc'"
10                 size="20"
11             ></icon>
12         </view>
13         <view class="rating-text">{{ avgRating.toFixed(1) }} ({{ ratingCount }}人评)
14     </view>
15     <view class="my-rating" v-if="myRating > 0">
16         <text class="my-rating-label">我的评分: </text>
17         <icon
18             v-for="(star, index) in 5"

```

```
18         :key="index"
19         :type="index < myRating ? 'star' : 'star-o'"
20         color="#ffcc33"
21         size="18"
22     ></icon>
23 </view>
24 <button v-else @click="showRatingModal" size="mini" plain>我要评分</button>
25
26
27 <uni-modal title="请评分" v-model="ratingModalShow" @confirm="submitRating">
28     <view class="modal-stars">
29         <icon
30             v-for="(star, index) in 5"
31             :key="index"
32             :type="index < selectedRating ? 'star' : 'star-o'"
33             :color="index < selectedRating ? '#ffcc33' : '#ccc'"
34             size="30"
35             @click="selectedRating = index + 1"
36         ></icon>
37     </view>
38 </uni-modal>
39 </view>
40 </template>
41
42 <script>
43 export default {
44     name: 'RatingComponent',
45     props: {
46         avgRating: {
47             type: Number,
48             default: 0
49         },
50         ratingCount: {
51             type: Number,
52             default: 0
53         },
54         myRating: {
55             type: Number,
56             default: 0
57         }
58     },
59     data() {
60         return {
61             ratingModalShow: false,
62             selectedRating: 3 // 默认选中3星
63         };
64     },
```



```

65     methods: {
66       showRatingModal() {
67         this.ratingModalShow = true;
68       },
69       submitRating() {
70         // 向父组件传递评分结果
71         this.$emit('rate', this.selectedRating);
72         this.ratingModalShow = false;
73       }
74     }
75   };
76 </script>

```

2.3.2 评分提交与数据同步

在分类列表页面中调用评分组件，处理评分提交逻辑，同步至服务端并更新页面状态：

Code block

```

1  // pages/category-list/category-list.vue
2  import RatingComponent from '@components/rating-component/rating-
   component.vue';
3
4  export default {
5    components: { RatingComponent },
6    methods: {
7      // 处理评分提交
8      async handleRate(item, rating) {
9        try {
10          // 1. 本地临时更新 (优化用户体验)
11          const oldRating = item.myRating;
12          item.myRating = rating;
13          // 计算新的平均评分 (服务端最终为准, 本地先预估)
14          const totalRating = item.avgRating * item.ratingCount - (oldRating ||
0) + rating;
15          item.avgRating = oldRating === 0 ? totalRating / (item.ratingCount +
1) : totalRating / item.ratingCount;
16          item.ratingCount = oldRating === 0 ? item.ratingCount + 1 :
item.ratingCount;
17          this.$forceUpdate();
18
19          // 2. 提交至服务端
20          const res = await this.$api.rateResource({
21            resourceId: item.id,
22            rating: rating,
23            oldRating: oldRating || 0
24          });

```

```

25
26     if (res.code === 200) {
27         // 3. 缓存个人评分记录
28         this.cacheMyRating(item.id, rating);
29         uni.showToast({ title: '评分成功' });
30     } else {
31         // 提交失败, 回滚本地状态
32         item.myRating = oldRating;
33         this.$forceUpdate();
34         uni.showToast({ title: res.msg || '评分失败', icon: 'none' });
35     }
36 } catch (err) {
37     // 网络异常, 本地缓存评分, 待网络恢复后同步
38     item.myRating = rating;
39     this.cacheMyRating(item.id, rating, true); // 第三个参数标记为待同步
40     this.$forceUpdate();
41     uni.showToast({ title: '网络异常, 评分已保存, 将在网络恢复后同步', icon:
'none' });
42 }
43 },
44 // 缓存个人评分记录
45 cacheMyRating(resId, rating, pending = false) {
46     const myRatings = uni.getStorageSync('myResourceRatings') || [];
47     const existIndex = myRatings.findIndex(item => item.resId === resId);
48     const ratingItem = { resId, rating, pending };
49     if (existIndex > -1) {
50         myRatings[existIndex] = ratingItem;
51     } else {
52         myRatings.push(ratingItem);
53     }
54     uni.setStorageSync('myResourceRatings', myRatings);
55 },
56 // 页面加载时同步个人评分
57 syncMyRatings() {
58     const myRatings = uni.getStorageSync('myResourceRatings') || [];
59     this.categoryData = this.categoryData.map(item => {
60         const myRatingItem = myRatings.find(r => r.resId === item.id);
61         if (myRatingItem) {
62             return { ...item, myRating: myRatingItem.rating };
63         }
64         return item;
65     });
66     // 同步待提交的评分 (网络恢复时调用)
67     this.syncPendingRatings();
68 },
69 // 同步待提交的评分
70 async syncPendingRatings() {

```

```
71     const myRatings = uni.getStorageSync('myResourceRatings') || [];  
72     const pendingRatings = myRatings.filter(item => item.pending);  
73     if (pendingRatings.length === 0) return;  
74  
75     for (const item of pendingRatings) {  
76         try {  
77             const res = await this.$api.rateResource({  
78                 resourceId: item.resId,  
79                 rating: item.rating,  
80                 oldRating: 0  
81             });  
82             if (res.code === 200) {  
83                 item.pending = false;
```

三、关键优化与注意事项

3.1 性能优化

- **避免重复请求：**下载前检查本地存储，评分前检查个人评分记录，减少无效接口调用。
- **批量同步：**待提交评分采用批量同步策略，避免频繁请求服务端。
- **懒加载与分页：**下载和评分功能不影响原有列表的分页加载逻辑，确保大量数据时页面流畅。

3.2 多端适配问题

- **文件路径差异：**不同平台的文件存储路径不同，统一使用`uni.env.USER\_DATA\_PATH`获取应用私有存储路径，避免权限问题。
- **进度条样式：**微信小程序与App端的progress组件样式有差异，通过自定义样式统一视觉效果。
- **图标兼容性：**使用Uni-app内置icon组件，避免第三方图标库的多端适配问题。

3.3 异常处理

- **下载失败：**提供重试按钮，记录失败原因，支持用户手动触发重新下载。
- **评分提交失败：**本地缓存评分记录，在页面重新加载且网络正常时自动同步。
- **文件损坏：**打开已下载文件失败时，提示用户重新下载，并清除无效的本地存储记录。

3.4 数据一致性保障

- **本地与服务端同步：**下载状态和评分以服务端数据为准，本地缓存仅用于优化体验，页面初始化时优先同步服务端数据。
- **跨页面状态同步：**通过页面栈获取分类列表页面实例，在“我的下载”页面删除资源时同步更新分类列表的下载状态。

四、扩展场景

- **下载队列管理**：支持多任务下载，展示下载队列，实现暂停/继续、优先级调整等功能。
- **评分评论联动**：在评分基础上新增评论功能，形成“评分+评论”的完整反馈体系。
- **资源更新提醒**：当已下载资源在服务端更新时，向用户推送更新提醒，支持增量更新。

五、总结

本次实现基于共用分类列表页面，通过“状态管理+本地缓存+组件封装”的方式，无缝集成了我的下载和评分功能。核心亮点在于：不破坏原有页面逻辑，保证多端适配一致性，通过本地缓存优化用户体验，同时通过异常处理和数据同步机制保障功能稳定性。开发过程中需重点关注多端文件路径差异、下载进度监听、评分数据一致性问题，确保功能在不同场景下均能正常运行。

（注：文档部分内容可能由 AI 生成）